

Summer Undergraduate Research Fellowship (SURF) Final Report

Decentralized, Browser-Based Docker IDEs for C++ and x86-64 Assembly

Education: Tested Across the Operating Systems and Processors Students

Own

Sean Cuenco

Department of Computer Science, University of Nevada, Las Vegas

Faculty Mentor: James Andro-Vasko, Ph.D., Assistant Professor in Residence and

Undergraduate Coordinator, Department of Computer Science

Office of Undergraduate Research, UNLV — Submitted October 1, 2026

1. Project Description

1.1 Main Objectives of the Research

This project did not begin as research. It began as a fix. UNLV computer science students in the introductory C++ course (CS 135) and the x86-64 assembly course (CS 218) lose the first week of the semester getting a compiler, an assembler, a debugger, and an editor working on dozens of different laptops, and students with Apple Silicon Macs often cannot run the course's virtual machine at all. Before the fellowship I built two Docker images to remove that week: each packs a browser-served Visual Studio Code editor (code-server) together with the exact toolchain the course expects, so a student installs only Docker Desktop, runs one command, and opens a browser tab. Everything runs on the student's own machine. The approach was chosen for its simplicity and ease of deployment, and it worked on the machine it was built on.

The Summer Undergraduate Research Fellowship turned that artifact into a research project by asking two questions the artifact could not answer on its own. First, is the promise

actually true? An environment that claims to give every student the same classroom has to be tested on the computers students actually own, not only on the developer's laptop. Second, what does this design give up compared with the ways other institutions have solved the same problem, from shared login servers and virtual machines to cloud-hosted editors and monitored classroom platforms? The fellowship's objectives followed from those two questions:

1. Measure, with one published test script run identically everywhere, whether the images behave the same across the operating systems (Windows, macOS, Linux) and processors (Intel/AMD and ARM, including Apple Silicon) that students bring to class: startup time, download size, memory, compile time under native execution versus emulation, debugger availability, and persistence of student files.
2. Preserve the courses' real x86-64 instruction set and Linux application binary interface (ABI) for assembly work while still supporting ARM hosts, and document precisely where that support ends.
3. Position the design within the research literature on standardized course environments and within UNLV's own history of student infrastructure, so that its trade-offs are stated rather than assumed.
4. Produce a study protocol for a future classroom evaluation, which is the only way to answer whether the environment changes setup time, support burden, or student experience.

1.2 Summary of the Problem and Why It Is Important

The problem is old, well documented, and expensive. Prior work has quantified the cost of the unproductive first week. Harvie et al. measured that software installation and troubleshooting consumed 75 to 100 minutes of class time per semester plus 65 to 90 minutes of

instructor troubleshooting before standardizing environments [6]. Valstar et al. found that 44% of their students owned Windows machines and 46% owned Macs while the autograder ran Linux, so a large share of support traffic was environment mismatch rather than course content [3]. Fernald et al. reported that students on M1 and M2 MacBooks could not run the x86 virtual machines their courses assumed at all [4].

UNLV has followed the same historical arc as the institutions in that literature, but has not yet adopted its most recent stage. For roughly two decades, UNLV computer science students reached course tools through shared departmental Linux login servers over SSH, commonly with PuTTY on Windows, using a submit script that required logging in to the server even from campus lab machines. Those servers (bobby, sally, and cardiac) have since been decommissioned; the documentation wiki that described them was deleted during a 2025 site rebuild, and the current guidance directs students to a VPN-gated login host and to centrally managed virtual desktop sessions whose local state is “PERMANENTLY DELETED” unless saved to a network drive [12]. For CS 218 specifically, the course distributes a VirtualBox virtual-machine image for Intel machines and an experimental, Fall-2023 UTM image for Apple Silicon that runs x86-64 under full-system emulation slow enough that the course page warns VS Code may be unusable; the primary recommendation for M1/M2 owners is to physically use the TBE-A311 lab with a USB drive. In other words, the environment problem at UNLV is real, it falls hardest on students with the newest laptops, and it currently affects several hundred CS 135 and CS 218 students each year.

Every institution in the literature has solved this problem by handing the setup burden to someone: to the department (shared servers), to the student’s hardware (virtual machines), to a cloud vendor and an account (Harvard’s Codespaces), or to the student’s privacy (monitored

classroom platforms). The question this project asks is whether a local container can leave the student carrying almost nothing after a single Docker installation, and whether that claim survives contact with real hardware. The problem matters beyond convenience. Setup friction is an equity issue (it penalizes students with older, cheaper, or less common hardware), a retention issue (the first weeks of an introductory course are when students decide whether they belong), and a pedagogical issue (time spent debugging an installation is time not spent learning pointers or the stack frame). Standardizing the environment is the precondition for measuring anything else about how students learn in these courses.

1.3 Specific Research Questions

Because the images had not yet been deployed in a full course section during the fellowship period, this cycle addressed the five research questions that can be answered before deployment, following the two-stage plan for a systems paper with a study protocol:

- **RQ1 (Reproducibility).** Do the published images execute the documented C++ and x86-64 student workflows, including real CS 218 assignments, successfully?
- **RQ2 (Portability).** Which combinations of host operating system and processor architecture can run the assembly environment, and what limitations remain?
- **RQ3 (Resource cost).** What storage, memory, startup-time, and compilation costs does the environment impose, and how large is the overhead of emulation on ARM hosts?
- **RQ4 (Persistence).** Does the documented bind-mount and update process preserve student files when the container or image is replaced?
- **RQ5 (Architecture fidelity).** To what extent can an amd64-only assembly environment reproduce the intended course behavior, including the debugger, on ARM hosts?

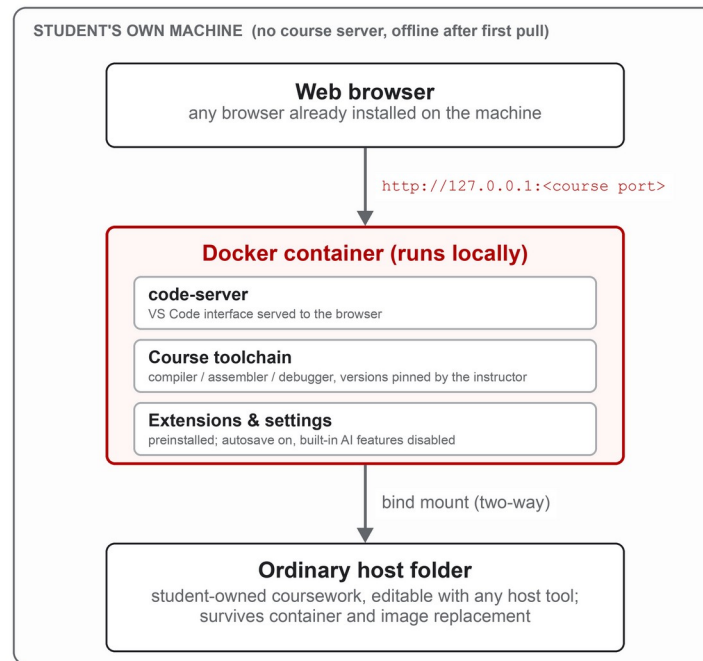
Five further questions on setup success rates, support burden, student experience, continued use, and learning outcomes (RQ6 to RQ10) are deferred to the classroom study described in Section 4.2.

2. Methodology

2.1 Design and Protocol Description

System design. Both images are built from Ubuntu 22.04 and publish code-server 4.126.0 on a localhost-only port whose number is the course number (8135 for C++, 8218 for x86). The C++ image ships g++ 11.4.0 and is a multi-architecture manifest that runs natively on both Intel/AMD (amd64) and ARM (arm64) hosts. The x86 image ships yasm 1.3.0, nasm 2.15.05, GNU ld, and gdb 12.1 and is deliberately amd64-only, because the course's .asm files can only assemble and execute as x86-64 Linux binaries; on ARM hosts Docker runs it under a translation layer. Student work lives in an ordinary folder on the host (a bind mount), never inside the container, and autosave writes it to disk as the student types, so a container can be deleted or replaced without touching coursework. Starter files are seeded only into an empty workspace so existing work is never overwritten. Containers carry no automatic restart policy, run under a minimal init process so they stop instantly, and disable every AI feature in the editor in keeping with course policy. Figure 1 shows the architecture. The Dockerfiles, handouts, and test scripts are public [11].

Figure 1. Architecture of one course IDE. Everything runs on the student’s own machine; the only network dependency is the initial image pull.



Evaluation design. The evaluation is a platform-matrix experiment. The same published shell script (`ci-test.sh`) is run, unmodified, on four host configurations, and every reported number is transcribed from the JSON file that script emits. The script performs nine stages in a fixed order: (1) a timed image pull recording the size and immutable digest; (2) a cold start timed until the editor answers its health endpoint; (3) a check that starter files are seeded exactly once; (4) three timed assemble-link-run cycles of the starter program; (5) a scripted gdb probe that sets a breakpoint, runs, and reads the instruction pointer register, recorded as working or broken; (6) idle container memory; (7) tool versions; (8) a timed build and execution of four real CS 218 assignments with peak container memory sampled throughout; and (9) destruction and recreation of the container to verify that student files survive and to time a warm start. Because the same digest is pulled everywhere, any difference between cells is attributable to the host, not the image.

Three cells were rented from public clouds so that the experiment could put a fresh, blank machine of each kind in the role of a student laptop, and one cell was an owned consumer laptop. Cloud instances were provisioned by hand, step by step, following runbooks written during the project, and torn down after each run. Windows Server on AWS proved to be outside Docker Desktop’s support matrix, so the Windows cell was moved to Azure, the only cloud that rents real Windows 11 Pro, which required upgrading a student subscription because the student-tier virtual machine sizes cannot nest virtualization and therefore cannot run WSL2. The total cloud spend for the whole matrix, including failed attempts, was on the order of a few dollars.

2.2 Materials, Sample Size, and Sampling Description

The unit of analysis is a host configuration, not a person; no human participants were involved and no student data were collected. The image under test was seancnc/unlv-x86-ide at a single pinned digest (sha256:1d0b91b3...), 552 MB, verified identical on every cell. The C++ image runs natively everywhere and was covered by the project’s separate release-check and continuous-integration workflow rather than the matrix, because a matrix row for it answers no research question. Table 1 lists the four cells.

Table 1. The platform matrix. Cells 1 and 2 are same-size Intel/Graviton siblings so their ratio isolates emulation.

Cell	Host	OS / Docker	CPU / RAM	x86 image mode
1 Linux amd64	AWS EC2 m8i.large	Ubuntu 24.04.4, Docker Engine 29.1.3	Intel Xeon 6975P-C, 2 vCPU, 8 GB	native
2 Linux arm64	AWS EC2 m8g.large (Graviton)	Ubuntu 24.04.4, Docker Engine 29.1.3, QEMU binfmt	Neoverse-V2, 2 vCPU, 8 GB	emulated (QEMU user-mode)
3 Windows amd64	Azure Standard_D4s_v5	Windows 11 Pro 24H2, Docker Desktop, WSL 2.7.11	Intel Xeon Platinum 8573C, 4 vCPU, 16 GB	native (inside WSL2)
4 macOS arm64	14-inch MacBook Pro (2021), owned	macOS 26.4, Docker Desktop 29.2.1	Apple M1 Pro, 8 cores, 16 GB	emulated (Rosetta via Docker Desktop)

The coursework workload consisted of four actual CS 218 assignments chosen to span the course's difficulty range: two pure-assembly programs (ast3 and ast04), a C++ driver calling assembly procedures with file I/O (ast06), and a multithreaded pthread program with an assembly kernel that computes a checkable numeric answer (ast12), the sharpest emulation-fidelity probe available. The workload manifest is public; the assignment sources are course material and are available on request. Cell 4 was deliberately the base-configuration 16 GB M1 Pro rather than the 64 GB development machine, because base-specification M1 Pros dominate the used market that budget-conscious students buy from. In addition to the matrix, the C++ image was used once in a three-student programming class for high school students as an informal pilot; no metrics or survey responses were collected from that use and it is reported only as a deployment fact.

Nine peer-reviewed papers were read in full and summarized in a shared annotated file, in a prescribed order: the direct lineage of containerized course environments [1, 3, 4], course-specific evidence on assembly and virtual machines [5, 6], the historical arc of local appliances [2, 7], and the contrast class of centralized monitoring and editor-embedded AI [8, 9]. A separate investigation, using live fetches, HTTP headers, DNS and SSH banner probes, an archived 2012 lab manual, and course Canvas pages supplied by the researcher, reconstructed the history of UNLV and College of Southern Nevada student infrastructure, with every claim labeled by evidence strength.

2.3 Analysis Framework and Technique

The technical analysis is descriptive and comparative. For each cell the script records pass/fail outcomes (seeding, persistence, per-assignment build and run status, gdb working or broken) and continuous measures (seconds and mebibytes). Native cells must pass every check;

on emulated cells a broken debugger or a failed assignment is recorded as a finding rather than a failure, because documenting the emulation boundary is the purpose of those cells. Emulation overhead is expressed as a ratio, emulated time divided by native time on the identical workload, with cell 1 as the denominator by design: cells 1 and 2 differ only in processor architecture (same cloud, generation, size, operating system, and Docker version), so the ratio isolates the translation layer. Cell 3 enters no ratio because it ran on a different cloud and instance size and is native anyway. Timer resolution is 0.1 s, so ratios are reported as order-of-magnitude figures and are only computed where the native denominator is non-zero.

Every anomaly was handled by a fixed rule: rerun to reproduce, capture diagnostics (container logs, kernel messages, memory, process state), archive everything verbatim, and report it. The literature was synthesized in a matrix comparing deployment location, isolation mechanism, student interface, persistence, internet dependence, evaluation evidence, and main limitation across sources, and the project was positioned as a point in that design space rather than as the first use of containers in teaching. Claims were sorted into three lists: supported by repository evidence, requiring measured technical evidence (this report), and requiring course data (future work), and the report makes no claim from the third list.

2.4 Specific Undergraduate Researcher (URM) Contributions to the Work

I designed and built both Docker images, their entrypoints, pinned editor settings, branding, starter files, and the student-facing Design Document and Update Instructions handouts, and I published the images to Docker Hub through a GitHub Actions pipeline that builds the C++ image natively for both architectures. I wrote the nine-stage measurement script and the workload manifest, and I chose the four CS 218 assignments and verified their expected outputs. I wrote the AWS and Azure runbooks, provisioned every cloud instance by hand

(console and command line), configured least-privilege roles, budgets, and security groups, diagnosed the failed launches, and personally ran the record runs, the Graviton crash reproduction, the QEMU follow-up experiment, and the Windows 11 cell. I read and annotated the nine papers, wrote the literature synthesis, carried out the institutional-history investigation including the DNS and SSH probes, and obtained the CS 218 Canvas evidence. Finally, I drafted the experiment plan, the results and evidence archive, the platform-findings analysis, this report, and the symposium poster. My mentor, Dr. Andro-Vasko, provided the course context and requirements for CS 135 and CS 218, guidance on which claims the evidence could support, and feedback on the research framing.

3. Major Findings

3.1 Results and Key Findings

Finding 1: the same image, byte for byte, behaved identically across three operating systems and two processor architectures, with one documented exception. Table 2 summarizes the record runs. The same 552 MB digest pulled on every cell. Starter seeding and persistence across container replacement passed on all four cells (RQ4). All four CS 218 assignments built and ran correctly on all four cells, including the multithreaded ast12 program producing its checkable answer under emulation (RQ1). The only behavioral difference is the debugger: gdb worked on both native cells and was broken on both emulated cells, exactly as predicted, and through two independent translation layers (QEMU user-mode on Linux, Apple's Rosetta path inside Docker Desktop on macOS). Under translation, a gdb script still appears to succeed and writes its output file, but the file contains only the labels with none of the register values, which is why the student handouts warn students not to trust an apparently successful debugger run on Apple Silicon (RQ5).

Table 2. Platform-matrix results for the x86 image (all numbers from the unmodified published script). Cell 3 timings are indicative only because it ran on a different cloud and instance size.

Metric	Cell 1 Linux amd64 (native)	Cell 2 Linux arm64 (QEMU)	Cell 3 Windows 11 amd64 (native)	Cell 4 macOS M1 Pro (emulated)
gdb probe	working	broken	working	broken
Pull time (s)	16.3	12.9	53.1	44.7
Image size (MB)	552	552	552	552
Cold start to healthy (s)	0.5	6.7	0.6	4.8
Warm start (s)	0.5	6.2	0.5	4.3
Starter assemble+link+run (s)	0.0	0.3	0.2	0.4
Idle memory (MiB)	54	255	56	263
Peak memory during coursework (MiB)	not captured	334	90	358
ast3 build / run (s)	0.0 / 0.0	0.3 / 0.1	0.2 / 0.2	0.4 / 0.2
ast04 build / run (s)	0.0 / 0.0	0.3 / 0.1	0.2 / 0.2	0.4 / 0.1
ast06 build / run (s)	0.2 / 0.0	2.8 / 0.1	0.5 / 0.2	2.5 / 0.2
ast12 build / run (s)	0.3 / 0.0	3.6 / 0.1	0.6 / 0.2	3.1 / 0.2
Starter seeding / persistence	pass / pass	pass / pass	pass / pass	pass / pass
Checks passed / failed	13 / 0	12 / 0	13 / 0	12 / 0

Finding 2: the resource cost is small on native hardware and modest under emulation (RQ3). On native hosts the editor is ready in about half a second and idles at roughly 55 MiB; the heaviest assignment builds in well under a second. Under emulation, startup rises to five to seven seconds, idle memory to about 255 to 265 MiB, and peak memory during the coursework to about 335 to 360 MiB. Even the emulated peak is a small fraction of an 8 GB laptop, which supports the claim that the environment is usable on the base-specification machines students actually own. Pull time was dominated by network rather than platform (13 to 53 s for the 552 MB image) and happens once.

Finding 3: emulation costs roughly one order of magnitude on compile-heavy steps, and the two translation layers cost about the same. On the two assignments with a usable

native denominator, QEMU user-mode emulation on Graviton was about 14 times slower than native on the ast06 build (2.8 s versus 0.2 s) and about 12 times slower on the ast12 build (3.6 s versus 0.3 s). Docker Desktop on the M1 Pro was about 12.5 times and 10 times slower on the same builds. In absolute terms every emulated build finished in under four seconds, so the overhead is measurable but not a barrier to coursework.

Finding 4: the support boundary is a translator boundary, not an operating-system boundary. The image runs wherever the layer that hosts Docker’s Linux environment either needs no translation or receives a high-quality x86 translator. Windows and Linux on Intel or AMD are native. Apple Silicon Macs work because Apple injects Rosetta into Docker’s Linux virtual machine. ARM Linux is the interesting case: under Ubuntu 24.04’s stock QEMU 8.2.2 the editor process crashed with an internal segmentation fault while translating code-server’s Node.js just-in-time compiler, reproduced on a second launch with diagnostics archived and memory exhaustion ruled out; the identical instance passed all twelve checks after switching to Docker’s own binfmt handler build. The crash is therefore bound to a translator version, not to the image or to ARM Linux, and the working one-line fix has been folded into the student documentation. Windows on ARM could not be tested in any cloud, because a rented virtual machine cannot nest the virtualization WSL2 requires on ARM sizes; it is expected to behave like cell 4 and is stated as an expectation, not a result. Intel Macs were likewise untested and are expected to behave as native amd64. Windows 11 was the strongest cell in the matrix: sub-second starts, 0.2 to 0.6 s builds, and a fully working debugger, which matters because Windows dominates student laptops.

Finding 5: the cloud is a workable, cheap stage for this kind of experiment, with one structural limit. Three of the four desired student environments could be summoned on demand

for cents to about a dollar per cell. Two rig failures produced no platform data and are reported as such: Windows Server 2025 on AWS silently refused Docker Desktop’s installer because Server is outside its support matrix, and an Ubuntu 26.04 variant never registered a QEMU handler. The methodological lesson for other educators is that a virtual-machine-based test rig cannot validate virtualization-dependent stacks on ARM; only physical hardware can close that cell.

Pilot deployment. The C++ image was used once in a three-student class for high school students. The environment ran on the students’ machines and the class proceeded, but no setup times, support counts, or survey responses were collected, so this use provides no evidence toward RQ6 to RQ10 and is reported only as a first deployment.

4. Broader Implications

4.1 Interpretation of the Findings in Relation to Related Studies

Harvard’s arc, and where this project sits on it. CS50 at Harvard is the best-documented history of a course environment: a shared Linux cluster, then server-side virtual machines on Amazon EC2, then the client-side CS50 Appliance virtual machine on students’ own laptops, then Docker containers behind a cloud IDE, and today VS Code on GitHub Codespaces used by more than 700,000 people [1, 2]. Malan’s own evaluation of the appliance era reads like a list of the problems containers were later built to solve: a nearly 2 GB image, hours to rebuild after any package update, nearly 20% of students finding it slow, and VirtualBox disk corruption when a laptop lid was closed [2]. Codespaces fixed those problems by moving the container to GitHub’s servers, but at the price of an account, a network connection, a provider relationship, and, by Malan’s own admission, students who finished the course “not really sure how to set up an IDE on my computer directly” [1]. The UNLV images

take the same container and the same browser-delivered VS Code and put the container back on the student's machine. The measurements in Section 3 are the evidence that the trade is affordable: starts in seconds rather than minutes, a 552 MB image rather than a 2 GB or 19 GB virtual disk, tens of mebibytes idle rather than a fixed 4 GB RAM reservation, and permanent student ownership of the environment with no offboarding problem. What the design gives up, and what CS50 keeps, is forced update convergence: a new image on Docker Hub only reaches students who pull it.

Table 3. Who carries the setup burden under each approach to a standardized course environment.

Approach	Who carries the burden	What it costs the student
Shared login servers over SSH (UNLV bobby/sally/cardiac; CSN bellagio)	The department: provisioning accounts, patching, keeping the server alive	Network and VPN dependence; loss of the environment and its documentation when servers are decommissioned
Course virtual machines (CS 218 VirtualBox .ova and UTM images; CS50 Appliance [2]; Reading's QEMU lab [5])	The student's hardware	Multi-gigabyte images, fixed RAM reservations, slow boots, disk corruption; unusable or unbearably slow on Apple Silicon
Cloud-hosted browser IDE (Harvard CS50 on GitHub Codespaces [1])	A vendor, a grant, and an account	Internet required; environment disappears with the account; students leave without knowing how to set up their own machine
Monitored classroom platform (CodeDive, South Korea [8])	The institution's cluster and the student's privacy	Every process and one-second typing pause recorded; consent that cannot realistically be withheld
Local container with browser editor (this project; cf. [3, 4])	The student, once: install Docker Desktop	One 552 MB download and one command; offline afterward; files stay on the laptop; debugger unavailable on Apple Silicon

The centralized alternative and the privacy trade. CodeDive, developed at Jeonbuk National University and partner universities in South Korea, is the modern centralized counterpart: browser VS Code in per-student containers on a Kubernetes cluster, with a kernel-level tracer that records every process start and a file watcher that snapshots a student's code after one second of typing pause [8]. In a two-week assignment with 95 students it captured

24,845 snapshots and used them for instructor dashboards and plagiarism screening. That is a rational response to assessment in the era of large language models, and it gives instructors visibility this project deliberately does not. The authors themselves note that such capture requires a data-protection framework under GDPR or FERPA that they have not yet built. The UNLV design occupies the opposite corner on purpose: disclosure is event-based and student-initiated (work exists only on the laptop until the student submits it), no telemetry is collected, and the position is enforced by architecture rather than policy. Similarly, CS50's guardrailed AI assistant hallucinated with confident tone and reached 88% accuracy on curricular questions [9]; the UNLV images disable every editor AI surface so that AI use is governed by course policy outside the environment rather than by an always-present assistant inside it. Both are considered positions in an active debate, not omissions.

Virtual machines, emulation, and the debugger trade-off. The virtual-machine literature frames the project's central design tension. Cadenas et al. replaced hardware boards with a QEMU full-system ARM emulator and measured a statistically significant rise in lab marks over six cohorts; crucially, they changed the course's instruction set to whatever the emulator provided, and in exchange kept gdb fully working inside the guest [5]. Harvie et al. hosted one virtual machine per student on department servers and recovered roughly half the class time previously lost to installation, but faculty had to build and maintain the server fleet themselves [6]. Laadan et al. showed in 2010 that a locally run appliance could support even kernel development for on-campus and remote students without a lab [7]. UNLV's own CS 218 materials exhibit the full-system corner of this space today: the official Apple Silicon route is a UTM image emulating a whole x86-64 machine, which keeps the debugger but is slow enough that the course warns the editor may be unusable. This project takes the opposite corner: user-

mode translation is roughly ten times slower than native on builds but still finishes in seconds, and it loses the debugger. Native amd64 gets both. No option on ARM hardware currently gets both, and the matrix makes that trade-off explicit and measured instead of implicit. This deliberate choice to preserve the course's real x86-64 ISA and ABI, rather than emulate a friendlier one, has no direct precedent in the nine papers reviewed, and the characterization of debugger behavior under user-mode emulation appears to be new within this literature.

Local containers in the literature, and the honest limitation. Valstar et al. (UC San Diego) and Fernalld et al. (Florida Tech) are the closest precedents for student-local containers. Seventy-one percent of UCSD students chose the DevContainer over the campus Linux server, and Florida Tech saw 84% adoption with 69% of respondents reporting reduced anxiety [3, 4]. Both also found the same choke point: installing Docker itself was the hardest step, and first-year students struggled most, with the high-anxiety share in Florida Tech's introductory course actually rising. The UNLV design reduces the steps after Docker (no desktop editor, no extension, no repository, one command and a browser tab) but inherits the Docker Desktop installation in full, and that should be stated plainly. Fernalld et al. also observed that M1 MacBooks silently built the wrong architecture, and their fix, centrally prebuilt and registry-hosted images, is the same distribution model used here.

UNLV's shared servers as motivation. The institutional investigation reframed the local motivation. The defensible claim is not that UNLV's old servers were merely outdated but that the shared-login model has churned opaquely: three named servers students built muscle memory around were decommissioned, the wiki documenting them was deleted and survives only in search-engine snippets, and one nominally decommissioned departmental login host was still answering SSH publicly in August 2026 with a 2016-era daemon (the hostname is withheld

here pending notification of campus IT). The current answer is a VPN-gated host plus centrally monitored virtual desktops with ephemeral local state. A local container is immune to every one of those failure modes: decommissioning, VPN dependence, documentation loss, and lost session state. That is a stronger and better-evidenced argument than staleness alone.

4.2 Potential Future Directions for Research

- **Classroom study (RQ6 to RQ10).** Deploy the x86 image in CS 218 and the C++ image in CS 135 with IRB approval, and measure the funnel from enrollment to attempted setup, successful setup, continued use, and survey response. Instruments are ready to be adapted from the literature: installation and troubleshooting minutes per lesson [6], support-forum traffic counts [3], and the pre/mid/post anxiety and setup-difficulty items calibrated against GAD-7 [4]. A survey draft was prepared during the fellowship. Any comparison of grades across semesters must be interpreted as association, not cause.
- **A hybrid multi-architecture assembly image.** The Graviton crash occurred while translating the editor, not the student's code. An arm64 variant could run code-server and the toolchain natively, use the x86 cross-assembler to emit genuine x86-64 binaries, and invoke the translator only to execute student programs, possibly recovering debugging through QEMU's built-in gdb stub. One variant would address ARM Linux and Windows on ARM at once, with no change to the student-facing command.
- **Closing the two untested cells with physical hardware.** An Intel Mac and a Snapdragon-based Windows laptop running the same published script would convert two stated expectations into results.
- **Generalizing the method.** The one-script, four-cell, cloud-rented platform matrix is course-agnostic. The same rig could evaluate environments for CS 370 (Operating

Systems), which as of its last public syllabus prescribed one of the decommissioned servers, and for other departments' tooling.

- **Broader consumer-hardware sampling.** Cell 4 is one machine. The classroom study should fold in the script's host record so that the distribution of real student hardware, and the timings on it, become data.

5. Reflection

5.1 Impact on Personal and Professional Development

This fellowship changed how I think about the difference between building something and knowing something about it. I came into the summer with working software: two Docker images that ran on my own machine and that I believed would run everywhere. The research process forced me to replace that belief with evidence, and the most important lesson was that a promise tested only where it succeeds is an advertisement, not a result. Designing the platform matrix taught me to identify the one variable I actually wanted to isolate (the emulation layer), to choose baselines so that nothing else changed, to treat a negative result as a finding rather than an embarrassment, and to archive every raw output so that someone else could check my numbers.

Reading the literature in a prescribed order, from Harvard's cluster-to-cloud-to-appliance history through the virtual-machine studies to the South Korean monitoring platform, gave me a vocabulary for what I had built and showed me that UNLV had followed the same arc as every other institution but stopped one stage early. That reframed the project from a tool I made into a position in a design space, with trade-offs I could name and defend: local versus cloud, telemetry versus privacy, instruction-set fidelity versus debugger availability. Learning to state what my

evidence could not support, and to separate design contributions from educational claims that only a classroom study can establish, was as valuable as any measurement.

Professionally, provisioning every cloud instance by hand before automating anything gave me a working command of AWS and Azure that I did not have in May: identity and access management, budgets, security groups, nested virtualization, subscription tiers, licensing constraints, and how to debug a Windows machine with no inbound network access. Writing runbooks that someone else could follow, and check-ins that my mentor could act on, made me a clearer technical writer. I also learned to communicate with a faculty mentor as a collaborator: to bring questions with proposed answers, to accept scope decisions, and to keep a written record of every decision and why it was made.

[Note to author: personalize this section before submission. Consider adding one concrete moment, for example the night the Graviton cell crashed and what you did next, and anything the weekly check-ins with Dr. Andro-Vasko taught you about presenting work in progress. Delete this note.]

5.2 Future Plans for Research Involvement

My immediate plan is to carry this project into its second stage: working with Dr. Andro-Vasko to deploy the images in CS 218 and CS 135, obtain IRB approval for the survey and setup-funnel instruments prepared this summer, and collect the classroom data that the pre-deployment results were designed to make interpretable. I intend to present the poster at the undergraduate research symposium and to prepare the system paper and study protocol for a computing-education venue such as SIGCSE or ITiCSE, where the papers that shaped this work were published. I also plan to build and test the hybrid arm64 assembly image described above, since it addresses the one platform gap the matrix left open.

Longer term, this fellowship confirmed that I want research to be part of my career. I am drawn to applied systems work that solves a concrete problem for a real population, measured honestly, and I plan to pursue graduate study in computer science with that focus. The Office of Undergraduate Research course modules on the research cycle and the mentor relationship gave me the framework; this project gave me the practice. I expect to keep using both.

References

- [1] David J. Malan. 2024. Containerizing CS50: Standardizing Students' Programming Environments. In Proceedings of the 2024 Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024). ACM, Milan, Italy.
<https://doi.org/10.1145/3649217.3653567>
- [2] David J. Malan. 2013. From Cluster to Cloud to Appliance. In Proceedings of the 18th ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '13). ACM, Canterbury, UK.
- [3] Sander Valstar, William G. Griswold, and Leo Porter. 2020. Using DevContainers to Standardize Student Development Environments: An Experience Report. In Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '20). ACM, Trondheim, Norway.
<https://doi.org/10.1145/3341525.3387424>
- [4] Kourtnee Fernalld, TJ OConnor, Sneha Sudhakaran, and Nasheen Nur. 2023. Lightweight Symphony: Towards Reducing Computer Science Student Anxiety with Standardized Docker Environments. In Proceedings of the 24th Annual Conference on Information Technology Education (SIGITE '23). ACM, Marietta, GA.
<https://doi.org/10.1145/3585059.3611432>

- [5] J. O. Cadenas, R. S. Sherratt, D. Howlett, C. G. Guy, and K. Lundqvist. 2015. Virtualization for Cost-Effective Teaching of Assembly Language Programming. *IEEE Transactions on Education* 58, 4 (2015), 282–288. <https://doi.org/10.1109/TE.2015.2405895>
- [6] David P. Harvie, Jason R. Cody, Christopher Morrell, and Tanya T. Estes. 2019. Using Virtual Machines to Enhance the Educational Experience in an Introductory Computing Course. In *Proceedings of the 20th Annual Conference on Information Technology Education (SIGITE '19)*. ACM, Tacoma, WA, 28–32. <https://doi.org/10.1145/3349266.3351401>
- [7] Oren Laadan, Jason Nieh, and Nicolas Viennot. 2010. Teaching Operating Systems Using Virtual Appliances and Distributed Version Control. In *Proceedings of the 41st ACM Technical Symposium on Computer Science Education (SIGCSE '10)*. ACM, Milwaukee, WI, 480–484.
- [8] H. Park, Y. Kim, K. Lee, S. Jin, J. Kim, Y. Heo, G. Kim, and E. Kim. 2025. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring for Programming Education. *Applied Sciences* 15, 19 (2025), 10403. <https://doi.org/10.3390/app151910403>
- [9] Rongxin Liu, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, and David J. Malan. 2024. Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*. ACM, Portland, OR. <https://doi.org/10.1145/3626252.3630938>
- [10] Ed Jorgensen. 2024. x86-64 Assembly Language Programming with Ubuntu (Version 1.1.58). University of Nevada, Las Vegas. <http://www.egr.unlv.edu/~ed/assembly64.pdf>

- [11] Sean Cuenco. 2026. UNLV Docker IDEs: source repository, handouts, measurement script, and record-run evidence. <https://github.com/seancnc003/UNLVDockerIDEs>. Images: hub.docker.com/r/seancnc/unlv-cpp-ide and hub.docker.com/r/seancnc/unlv-x86-ide.
- [12] UNLV Department of Computer Science. 2025. Student Center: Remote Access and File Storage. <https://tux.cs.unlv.edu/remote-access.html> (accessed August 16, 2026).